UNIVERSIDADE FEDERAL DE SERGIPE

Programação dinâmica

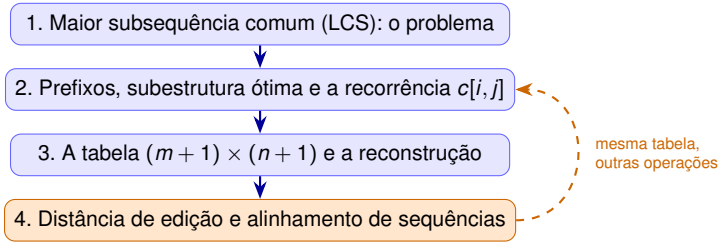
Maior subsequência comum e distância de edição

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

20 de outubro de 2026

O que você vai aprender hoje



Na mochila, os dois índices eram *item* e *capacidade*. Hoje são duas **posições**, uma em cada sequência, e o tempo $\Theta(mn)$ é polinomial de verdade.

Maior subsequência comum

Por que comparar duas sequências?

Problema:

Quanto duas sequências se parecem, e onde elas concordam?

- `diff` e `git`: que linhas de um arquivo foram mantidas na nova versão?
- Bioinformática: dois trechos de DNA (`ACCGGTCG...`) vêm de um ancestral comum?
- Corretor ortográfico: qual palavra do dicionário está mais perto de "`algoritimo`"?

Solução:

Medir a semelhança pela **maior subsequência comum**: o maior conjunto de símbolos que aparecem nas duas sequências *na mesma ordem*, não necessariamente colados.

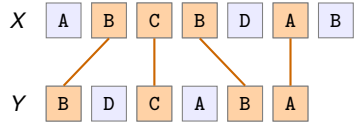
A força bruta é exponencial. A programação dinâmica resolve em $\Theta(mn)$, com os mesmos quatro passos de sempre: estrutura, recorrência, tabela, reconstrução.

Para aprofundar: Cormen et al. (2009), §15.4 [1]; Hunt e McIlroy (1976) [5].

Subsequência e LCS: definição

Definição

$Z = \langle z_1, \ldots, z_k \rangle$ é **subsequência** de $X = \langle x_1, \ldots, x_m \rangle$ se existem índices $i_1 < i_2 < \cdots < i_k$ com $x_{i_j} = z_j$. Uma **subsequência comum** de X e Y é subsequência das duas; o problema da **LCS** pede uma de comprimento máximo.



$\langle B, C, B, A \rangle$: comprimento 4,
uma LCS (não é a única: $\langle B, D, A, B \rangle$ também é)

Subsequência *não* é subcadeia: os símbolos podem ter "buracos" entre si, mas as linhas que os ligam nunca se cruzam.

Para aprofundar: Cormen et al. (2009), §15.4, exemplo da Figura 15.8 [1].

Força bruta: 2^m candidatos

Enumerar todas as subsequências de X e testar cada uma contra Y:

- cada subconjunto de índices de X é uma subsequência: 2^m candidatos;
- testar se Z é subsequência de Y custa $O(n)$, com uma varredura gulosa (Exercício 1 do tutorial);
- total $O(n\,2^m)$.

Para comparar dois genes com $m = n = 1000$ bases, $2^{1000} \approx 10^{301}$ candidatos. A PD fará 10^6 passos.

$m = n$	força bruta $n\,2^m$	PD mn
10	$\approx 10^4$	10^2
30	$\approx 3 \cdot 10^{10}$	$9 \cdot 10^2$
60	$\approx 7 \cdot 10^{19}$	$3{,}6 \cdot 10^3$
1000	$\approx 10^{304}$	10^6

Para aprofundar: Cormen et al. (2009), §15.4, “Step 1” [1].

Recapitulando: o problema da LCS

O que aprendemos

- Subsequência: símbolos de X na mesma ordem, com buracos permitidos.
- LCS: subsequência comum de comprimento máximo; pode haver várias.
- No exemplo do Cormen, $X = \texttt{ABCBDAB}$ e $Y = \texttt{BDCABA}$ têm LCS de comprimento 4, por exemplo `BCBA`.
- A força bruta testa 2^m subsequências: inviável já para $m \approx 60$.

Para aprofundar: Cormen et al. (2009), §15.4 [1].

Formulação

Por que prefixos?

Problema:

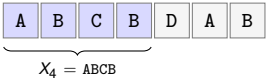
Para aplicar PD, precisamos de uma família pequena de subproblemas do *mesmo tipo*. Pares arbitrários de subsequências de X e Y são $2^m \cdot 2^n$: não serve.

Solução:

Olhar para o **último símbolo** de cada sequência, como olhávamos para o último item da mochila. Decidir sobre x_m e y_n deixa sempre um par de **prefixos**:

$$X_i = \langle x_1, \ldots, x_i \rangle, \qquad Y_j = \langle y_1, \ldots, y_j \rangle.$$

São só $(m+1)(n+1)$ pares (X_i, Y_j).



o prefixo X_0 é a sequência vazia

Para aprofundar: Cormen et al. (2009), §15.4, "Step 1: Characterizing a longest common subsequence" [1].

Subestrutura ótima da LCS

Teorema (Cormen et al., Teorema 15.1)

Seja $Z = \langle z_1, \ldots, z_k \rangle$ uma LCS de X_m e Y_n.

1. Se $x_m = y_n$, então $z_k = x_m = y_n$ e Z_{k-1} é LCS de X_{m-1} e Y_{n-1}.
2. Se $x_m \neq y_n$ e $z_k \neq x_m$, então Z é LCS de X_{m-1} e Y_n.
3. Se $x_m \neq y_n$ e $z_k \neq y_n$, então Z é LCS de X_m e Y_{n-1}.

Ideia da prova (caso 1, recorta e cola). Se $z_k \neq x_m$, acrescentar $x_m = y_n$ ao fim de Z daria uma subsequência comum de comprimento $k+1$: absurdo. Logo $z_k = x_m$. Se houvesse uma subsequência comum W de X_{m-1} e Y_{n-1} maior que Z_{k-1}, então W seguida de x_m seria maior que Z: absurdo de novo. Os casos 2 e 3: Z não usa o último símbolo de uma das sequências, então é subsequência comum do par menor; se não fosse a maior lá, também não seria aqui.

Para aprofundar: Cormen et al. (2009), §15.4, Teorema 15.1 [1].

A recorrência

Definição

$c[i, j]$ = comprimento de uma LCS de X_i e Y_j. A resposta é $c[m, n]$.

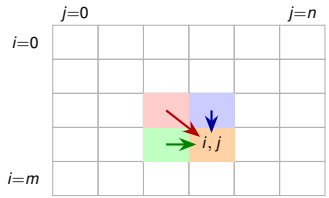
$$c[i,j] = \begin{cases} 0 & \text{se } i = 0 \text{ ou } j = 0, \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j, \\ \max\big(\underbrace{c[i-1,j]}_{\text{descarta } x_i}, \underbrace{c[i,j-1]}_{\text{descarta } y_j}\big) & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases}$$

Intuição

Se os últimos símbolos são iguais, vale a pena casá-los (Teorema 15.1, caso 1). Se são diferentes, pelo menos um deles fica de fora da LCS; como não sabemos qual, testamos os dois, exatamente como na mochila (item dentro ou fora).

Para aprofundar: Cormen et al. (2009), §15.4, equação (15.9) [1].

Grafo de subproblemas e ordem de preenchimento



A célula (i, j) depende de três vizinhas:

- $(i-1, j-1)$, a diagonal, quando $x_i = y_j$;
- $(i-1, j)$, acima, e $(i, j-1)$, à esquerda, quando $x_i \neq y_j$.

Preencher **linha por linha, da esquerda para a direita** respeita todas as dependências. Tempo $\Theta(mn)$, espaço $\Theta(mn)$.

A recursão direta, sem tabela, repete subproblemas: no exemplo do Cormen ($m = 7$, $n = 6$) já são 152 chamadas para 42 células, e o número cresce exponencialmente quando X e Y não têm símbolos em comum.

Para aprofundar: Cormen et al. (2009), §15.4, "Step 3" [1].

Recapitulando: formulação

O que aprendemos

- Subproblemas: pares de prefixos (X_i, Y_j), só $(m+1)(n+1)$ deles.
- Subestrutura ótima (Teorema 15.1): ou os últimos símbolos casam, ou um deles fica de fora.
- Recorrência: $c[i,j] = c[i-1,j-1]+1$ se $x_i = y_j$; senão $\max(c[i-1,j],\ c[i,j-1])$.
- Cada célula depende de três vizinhas (acima, esquerda, diagonal); linha por linha dá $\Theta(mn)$.

Para aprofundar: Cormen et al. (2009), §15.4 [1].

Tabela e reconstrução

Bottom-up em C

Exemplo condutor (Cormen, Figura 15.8):

$X = \texttt{ABCBDAB}$, $m = 7$
$Y = \texttt{BDCABA}$, $n = 6$

As cadeias são indexadas a partir de 1 nos slides; em C, x_i é `x[i-1]`.

Os dois laços preenchem $(m+1)(n+1) = 56$ células, cada uma em $O(1)$.

```c
/* c[i][j]: comprimento de uma LCS de x[0..i-1] e y[0..j-1] */
int lcs( const char x[], int m, const char y[], int n) {
   for (int i = 0; i <= m; ++i) c[i][0] = 0;
   for (int j = 0; j <= n; ++j) c[0][j] = 0;
   for (int i = 1; i <= m; ++i)
      for (int j = 1; j <= n; ++j)
         if (x[i-1] == y[j-1])
            c[i][j] = c[i-1][j-1] + 1;      /* casa x_i com y_j */
         else if (c[i-1][j] >= c[i][j-1])
            c[i][j] = c[i-1][j];            /* descarta x_i     */
         else
            c[i][j] = c[i][j-1];            /* descarta y_j     */
   return c[m][n];
}
```

Para aprofundar: Cormen et al. (2009), §15.4, LCS-LENGTH [1].

Exemplo: a tabela c preenchida

		B	D	C	A	B	A
	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

Para aprofundar: Cormen et al. (2009), §15.4, Figura 15.8 [1].

Célula $(6,6)$: $x_6 = \mathtt{A} = y_6$, diagonal:

$$c[6,6] = c[5,5] + 1 = 3 + 1 = 4.$$

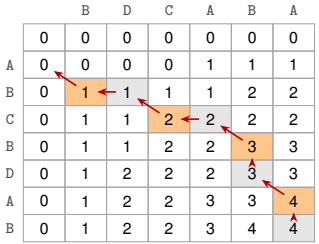
Célula $(5,5)$: $x_5 = \mathtt{D} \neq \mathtt{B} = y_5$:

$$c[5,5] = \max(\underbrace{3}_{c[4,5]}, \underbrace{2}_{c[5,4]}) = 3.$$

Linha 0 e coluna 0: um prefixo vazio, LCS vazia.

Resposta: $c[7,6] = 4$.

Reconstruindo uma LCS



		B	D	C	A	B	A
	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

```c
/* imprime uma LCS de x[0..i-1] e y[0..j-1] */
void imprime( const char x[], const char y[], int i, int j)
      {
   if (i == 0 || j == 0) return;
   if (x[i-1] == y[j-1]) {
      imprime( x, y, i-1, j-1);
      putchar( x[i-1]);               /* casou */
   } else if (c[i-1][j] >= c[i][j-1])
      imprime( x, y, i-1, j);         /* sobe  */
   else
      imprime( x, y, i, j-1);         /* esquerda */
}
```

Partindo de $(7, 6)$, cada passo refaz a decisão da recorrência. As células laranja são os casamentos: `B`, `C`, `B`, `A`. Custo $O(m + n)$.

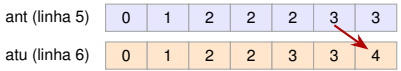
Para aprofundar: Cormen et al. (2009), §15.4, PRINT-LCS e Figura 15.8 [1].

Economia de espaço

Para obter só o **comprimento** $c[m, n]$, a linha i depende apenas da linha $i - 1$:

- duas linhas bastam: espaço $\Theta(n)$;
- escolhendo a sequência menor como colunas: $\Theta(\min(m, n))$ (Cormen, Exercício 15.4-4);
- o tempo continua $\Theta(mn)$.

Mas sem a tabela inteira a reconstrução da página anterior não funciona. Hirschberg (1975) combina PD com divisão e conquista e reconstrói a LCS em espaço linear, gastando só o dobro do tempo.



ao fim de cada linha, `ant` e `atu` trocam de papel

Para aprofundar: Cormen et al. (2009), Exercício 15.4-4 [1]; Hirschberg (1975) [4]; Kleinberg e Tardos (2006), §6.7 [6].

Recapitulando: tabela e reconstrução

O que aprendemos

- Bottom-up: dois laços aninhados sobre $(m+1)(n+1)$ células, $\Theta(mn)$.
- No exemplo do Cormen, $c[7,6] = 4$ e a reconstrução encontra `BCBA`.
- Reconstrução em $O(m+n)$: casou, sobe ou vai à esquerda, refazendo a recorrência a partir de (m,n).
- Só o comprimento: espaço $\Theta(\min(m,n))$; a sequência em espaço linear exige o algoritmo de Hirschberg.

Para aprofundar: Cormen et al. (2009), §15.4 [1].

Distância de edição

Por que medir distância entre cadeias?

Problema:

O usuário digita "`algoritimo`". Que palavra ele quis escrever? Um gene sofreu mutações: quantas, no mínimo?

A LCS só diz o que *coincide*. Queremos contar o que precisa **mudar**, e uma troca de letra deve custar menos que apagar uma e inserir outra.

Solução:

A **distância de edição** (Levenshtein, 1966): o menor número de inserções, remoções e substituições de um símbolo que transformam X em Y.

Wagner e Fischer (1974) deram a PD: a mesma tabela de prefixos da LCS, com $\min$ no lugar de $\max$ e uma terceira opção, a substituição.

Para aprofundar: Levenshtein (1966) [7]; Wagner e Fischer (1974) [9]; Dasgupta et al. (2008), §6.3 [3].

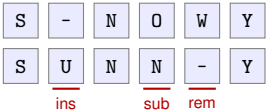
Distância de edição: definição

Definição

A **distância de edição** $d(X, Y)$ é o menor custo de uma sequência de operações que transforma X em Y, cada uma de custo 1: **inserir** um símbolo, **remover** um símbolo ou **substituir** um símbolo por outro.

Intuição: alinhamento

Escreva X sobre Y, com lacunas (`-`). Cada coluna é um casamento (custo 0), uma substituição, uma remoção (`-` embaixo) ou uma inserção (`-` em cima). O custo do alinhamento é o número de colunas diferentes.



$d(\texttt{SNOWY}, \texttt{SUNNY}) = 3$:
insere `U`, troca `O` por `N`, remove `W`

Para aprofundar: Dasgupta et al. (2008), §6.3 [3]; Levenshtein (1966) [7].

A recorrência da distância de edição

$E[i, j]$ = distância de edição entre os prefixos X_i e Y_j. A **última coluna** de um alinhamento ótimo é uma de três:

$$E[i,j] = \min \begin{cases} E[i-1,j] + 1 & x_i \text{ sobre } - \text{ (remove } x_i\text{)}, \\ E[i,j-1] + 1 & - \text{ sobre } y_j \text{ (insere } y_j\text{)}, \\ E[i-1,j-1] + \mathrm{dif}(x_i, y_j) & x_i \text{ sobre } y_j, \end{cases}$$

com $\mathrm{dif}(a, b) = 0$ se $a = b$ e 1 caso contrário.

Casos base: $E[i, 0] = i$ (remover tudo) e $E[0, j] = j$ (inserir tudo).

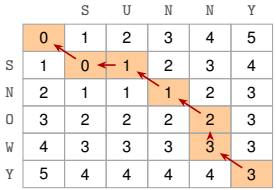
$\cdots$ $\begin{matrix} x_i \\ - \end{matrix}$ $(i-1, j)$

$\cdots$ $\begin{matrix} - \\ y_j \end{matrix}$ $(i, j-1)$

$\cdots$ $\begin{matrix} x_i \\ y_j \end{matrix}$ $(i-1, j-1)$

Para aprofundar: Dasgupta et al. (2008), §6.3 [3]; Wagner e Fischer (1974) [9]; Cormen et al. (2009), Problema 15-5 [1].

Exemplo: a tabela de SNOWY para SUNNY



		S	U	N	N	Y
	0	1	2	3	4	5
S	1	0	1	2	3	4
N	2	1	1	1	2	3
O	3	2	2	2	2	3
W	4	3	3	3	3	3
Y	5	4	4	4	4	3

Célula $(3,4)$, $x_3 = \texttt{O}$, $y_4 = \texttt{N}$:

$$E[3,4] = \min(\underbrace{2{+}1}_{\text{remove}}, \underbrace{2{+}1}_{\text{insere}}, \underbrace{1{+}1}_{\text{troca}}) = 2.$$

Caminho de volta a partir de $(5,5)$, lido de trás para frente:

- diagonal com custo 0: casamento (S, N, Y);
- diagonal com custo 1: substituição (O→N);
- para cima: remove W; para a esquerda: insere U.

Resposta: $E[5,5] = 3$, o alinhamento da página anterior.

Para aprofundar: Dasgupta et al. (2008), §6.3, Figura 6.4 [3].

Distância de edição em C

```c
/* E[i][j]: distancia de edicao entre x[0..i-1] e y[0..j-1] */
int edicao( const char x[], int m, const char y[], int n) {
   for (int i = 0; i <= m; ++i) E[i][0] = i;   /* remove tudo */
   for (int j = 0; j <= n; ++j) E[0][j] = j;   /* insere tudo */
   for (int i = 1; i <= m; ++i)
      for (int j = 1; j <= n; ++j) {
         int r = E[i-1][j] + 1;                          /* remove x_i */
         if (E[i][j-1] + 1 < r) r = E[i][j-1] + 1;       /* insere y_j */
         int d = E[i-1][j-1] + (x[i-1] != y[j-1]);       /* casa/troca */
         E[i][j] = d < r ? d : r;
      }
   return E[m][n];
}
```

A estrutura é a do `lcs`: só mudam os casos base, o `min` e o custo da diagonal.

Tempo e espaço $\Theta(mn)$; com duas linhas, espaço $\Theta(\min(m, n))$ (Exercício 4 do tutorial).

A reconstrução do alinhamento segue o mesmo padrão: de (m, n) até $(0, 0)$, escolhendo a vizinha que produziu o mínimo.

Para aprofundar: Wagner e Fischer (1974) [9]; Dasgupta et al. (2008), §6.3 [3].

LCS e edição: a mesma tabela

Proíba a substituição: só inserções e remoções. Num alinhamento, as colunas com casamento formam uma subsequência comum; todo o resto é lacuna. Então

$$d_{\text{ins/rem}}(X, Y) = m + n - 2\,|\mathrm{LCS}(X, Y)|.$$

- Cormen: $7 + 6 - 2 \cdot 4 = 5$.
- `SNOWY`/`SUNNY`: LCS `SNY`, $5 + 5 - 2 \cdot 3 = 4$, contra 3 quando a substituição é permitida.

É essa a distância que o `diff` mede entre dois arquivos, tomando cada **linha** como um símbolo.

	LCS	edição
objetivo	max	min
casos base	0	i e j
diagonal	se $x_i = y_j$	sempre, $+\mathrm{dif}$
acima/esq.	$+0$	$+1$
tempo	$\Theta(mn)$	$\Theta(mn)$

Para aprofundar: Cormen et al. (2009), §15.4 e Problema 15-5 [1]; Hunt e McIlroy (1976) [5].

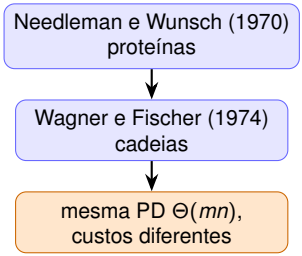
Alinhamento de sequências com custos

Em biologia, nem toda diferença custa igual: trocar `A` por `G` (ambas purinas) é mais comum que trocar `A` por `C`. O alinhamento de Kleinberg e Tardos usa

- δ: custo de cada lacuna;
- α_{pq}: custo de alinhar p com q ($\alpha_{pp} = 0$).

$$\mathrm{OPT}(i,j) = \min \begin{cases} \alpha_{x_i y_j} + \mathrm{OPT}(i-1, j-1), \\ \delta + \mathrm{OPT}(i-1, j), \\ \delta + \mathrm{OPT}(i, j-1). \end{cases}$$

Com $\delta = 1$ e $\alpha_{pq} = 1$ para $p \neq q$, é a distância de edição.



Para aprofundar: Kleinberg e Tardos (2006), §6.6 [6]; Needleman e Wunsch (1970) [8].

Recapitulando: distância de edição

O que aprendemos

- Distância de edição: mínimo de inserções, remoções e substituições; $d(\texttt{SNOWY}, \texttt{SUNNY}) = 3$.
- Recorrência pela última coluna do alinhamento: remove, insere ou casa/troca.
- Mesma tabela de prefixos da LCS, com $\min$, casos base i e j e custo na diagonal; $\Theta(mn)$.
- Sem substituição, $d = m + n - 2|\mathrm{LCS}|$; com custos δ e α, é o alinhamento usado em bioinformática.

Para aprofundar: Dasgupta et al. (2008), §6.3 [3]; Kleinberg e Tardos (2006), §6.6 [6].

Fechando o ciclo

	intervalos	mochila 0/1	LCS	edição
subproblema	$\mathrm{OPT}(j)$	$\mathrm{OPT}(i, c)$	$c[i, j]$	$E[i, j]$
última decisão	j entra?	i entra?	$x_i = y_j$?	última coluna
dependências	2	2	3	3
tabela	vetor	$(n+1) \times (W+1)$	$(m+1) \times (n+1)$	$(m+1) \times (n+1)$
tempo	$O(n \log n)$	$\Theta(nW)$, pseudopol.	$\Theta(mn)$	$\Theta(mn)$

Próxima aula: Warshall e Floyd. A PD agora trabalha sobre grafos: o subproblema é “caminhos que usam só os vértices $1, \ldots, k$ como intermediários”.

Para estudar: Cormen, §15.4 e Problema 15-5; Dasgupta, §6.3; Kleinberg e Tardos, §6.6. O desafio do tutorial é o alinhamento de DNA com custos.

Para aprofundar: Cormen et al. (2009), §§15.4 e 25.2 [1]; Kleinberg e Tardos (2006), §6.6 [6].

Referências I



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
Algorithms.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



Daniel S. Hirschberg.
A linear space algorithm for computing maximal common subsequences.
Communications of the ACM, 18(6):341–343, 1975.



James W. Hunt and M. Douglas McIlroy.
An algorithm for differential file comparison.
Computing Science Technical Report 41, Bell Laboratories, 1976.

Referências II



Jon Kleinberg and Éva Tardos.
Algorithm Design.
Pearson/Addison-Wesley, 2006.
Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Vladimir I. Levenshtein.
Binary codes capable of correcting deletions, insertions, and reversals.
Soviet Physics Doklady, 10(8):707–710, 1966.



Saul B. Needleman and Christian D. Wunsch.
A general method applicable to the search for similarities in the amino acid sequence of two proteins.
Journal of Molecular Biology, 48(3):443–453, 1970.



Robert A. Wagner and Michael J. Fischer.
The string-to-string correction problem.
Journal of the ACM, 21(1):168–173, 1974.